Sprawozdanie – Laboratorium 4: Authentication, authorization i konfiguracja

Przedmiot: Zaawansowane Technologie Programowania Aplikacji Internetowych

Temat: Room Booking API (Role, bezpieczeństwo, OpenAPI)

Student: Kacper Gumulak – 151872

GitHub: github.com/zephir-x/pk-ztpai

1. Krótki opis rozwiązania

Zadanie polegało na rozbudowie aplikacji "Room Booking API" o system uwierzytelniania (Authentication) oraz autoryzacji (Authorization). Dodano pełną warstwę zarządzania użytkownikami z wykorzystaniem algorytmu bcryptjs do bezpiecznego hashowania haseł z użyciem tzw. soli. Komunikacja bezstanowa została zrealizowana w oparciu o tokeny JWT (jsonwebtoken).

Wprowadzono system kontroli dostępu na dwóch poziomach:

1. **RBAC (Role-Based Access Control):** Oparty na middleware authorize, chroniący zasoby globalne (np. tworzenie budynków i sal jest dostępne tylko dla konta ADMIN).
2. **Ownership (Kontrola własności):** Zaimplementowany głęboko w warstwie serwisu (reservations.service.js).

Uzasadnienie decyzji dla Endpointu GET /api/reservations: Zgodnie z wymaganiami, dla użytkownika z rolą USER zignorowano ewentualny parametr query ?userId=X i zamiast zwracać błąd 403 (Forbidden), nadpisano go bezpiecznie na backendzie identyfikatorem z tokenu (userId = currentUser.id). Takie podejście jest bardziej *robuste* – zapobiega wyciekowi informacji, uniemożliwia błędy po stronie frontendu i gwarantuje, że zwykły użytkownik zawsze otrzyma wyłącznie własne dane, niezależnie od prób manipulacji adresem URL. Z kolei przy próbie odczytu lub anulowania **pojedynczej** obcej rezerwacji przez użytkownika USER, system zwraca błąd 404 Not Found. Decyzja ta ukrywa istnienie prywatnych zasobów innych osób (Security through obscurity), co jest lepszą praktyką niż zwracanie 403 Forbidden zdradzającego, że dany zasób o podanym ID faktycznie istnieje.

Zabezpieczono również proces uruchamiania aplikacji poprzez wdrożenie wzorca *Fail-Fast* – scentralizowany moduł weryfikuje obecność i poprawność zmiennych środowiskowych przy użyciu biblioteki Zod, zapobiegając uruchomieniu serwera bez podanego klucza JWT_SECRET. Dodatkowo, wystawiono publicznie interaktywną dokumentację w standardzie OpenAPI przy pomocy swagger-ui-express. Zbadano również historię Git poleceniem git log -p --all | grep -iE "JWT_SECRET=|password=" – potencjalne wycieki sekretów z wczesnych faz developmentu uznano za skompromitowane, a klucze obrócono (wygenerowano nowe) w środowisku lokalnym.

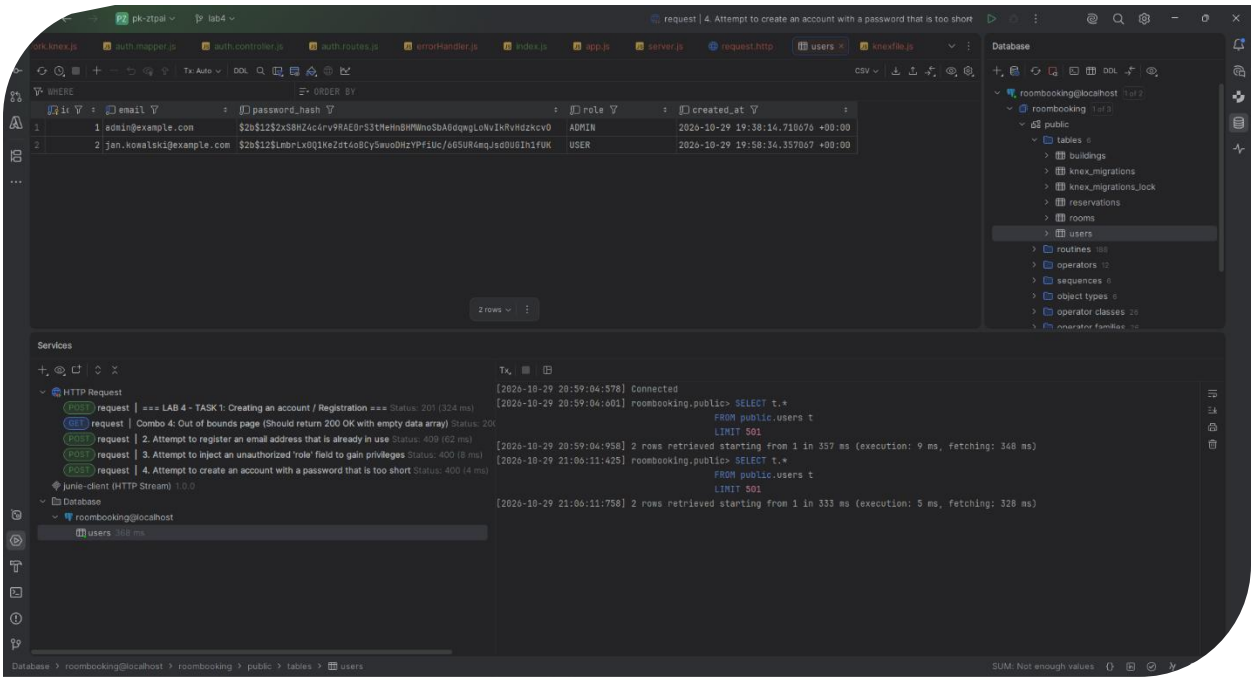
2. Realizacja zadań

Zadanie 1: Rejestracja

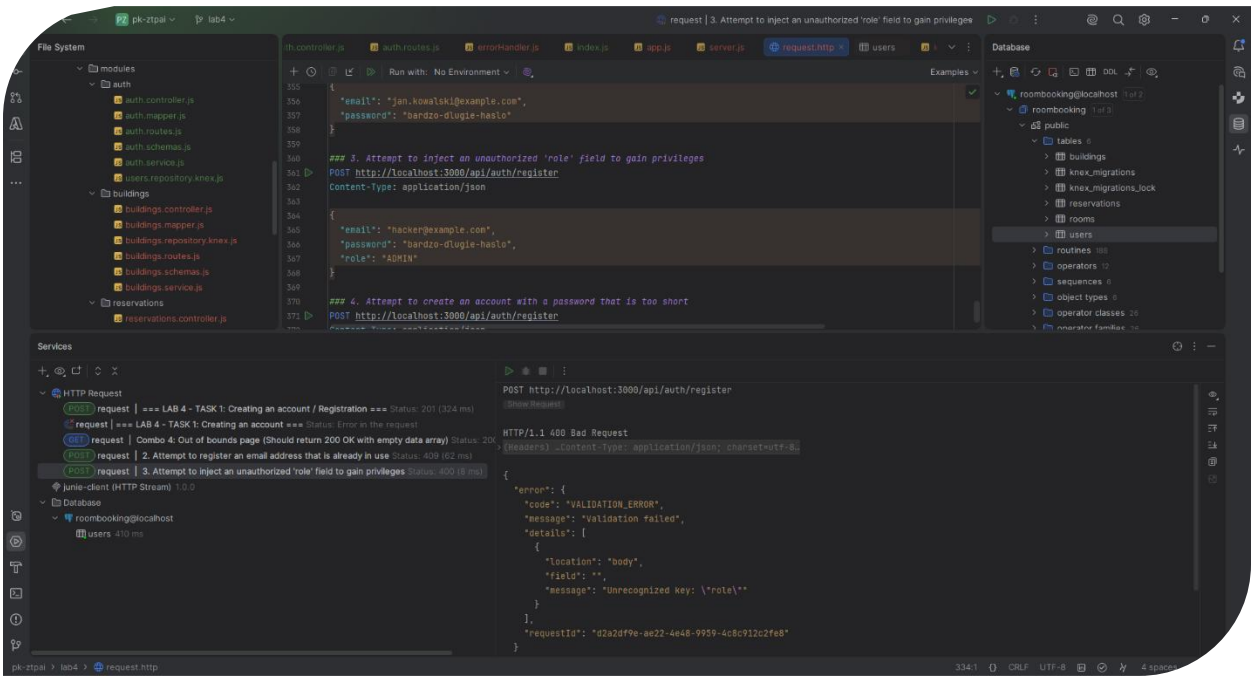
Zaimplementowano bezpieczny endpoint do tworzenia użytkowników z ujednoliconą obsługą błędów. Unikalność adresu e-mail jest gwarantowana przez constraint bazy danych (aby uniknąć *race conditions*), a błąd 23505 jest mapowany na aplikacyjny kod 409 Conflict. Użytkownik domyślnie otrzymuje rolę USER.

Fragment kodu (Serwis autoryzacyjny – hashowanie i rejestracja):

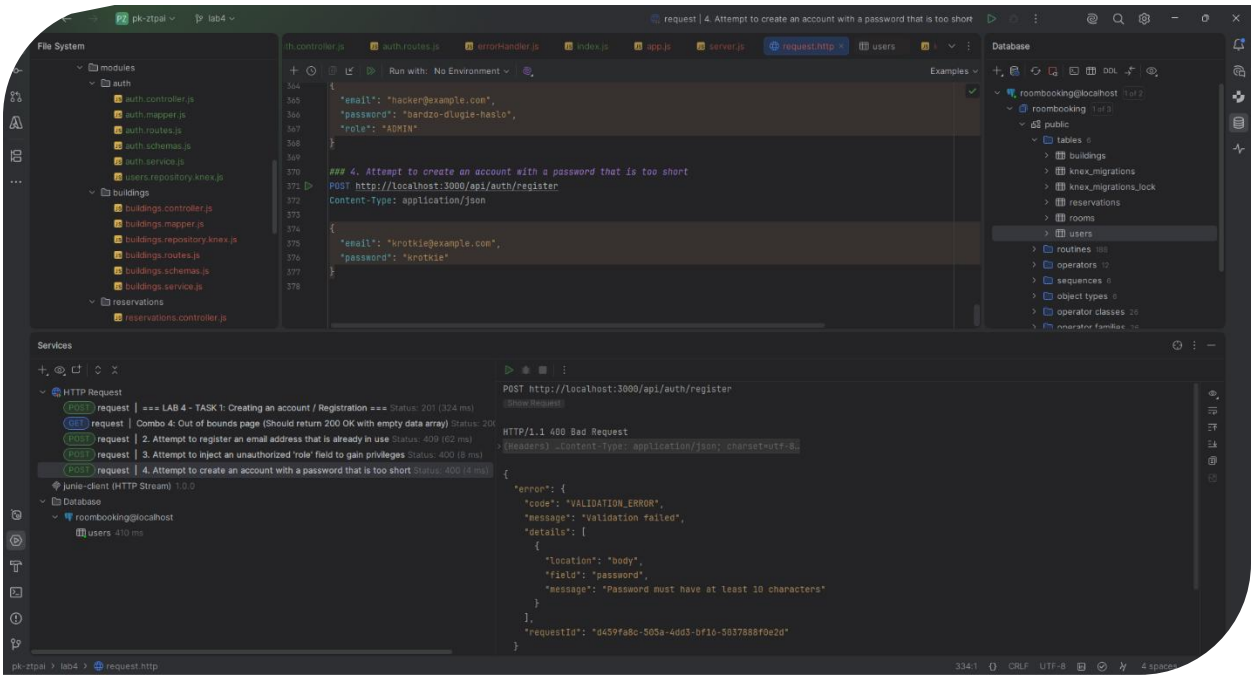
```
async function register({ email, password }) {
    const existing = await usersRepository.findByEmail(email);
    if (existing) {
        throw new ConflictError("Email already registered", "EMAIL_TAKEN");
    }
    const passwordHash = await bcrypt.hash(password, 12); // Cost factor 12
    return usersRepository.create({ email, passwordHash, role: "USER" });
}
```



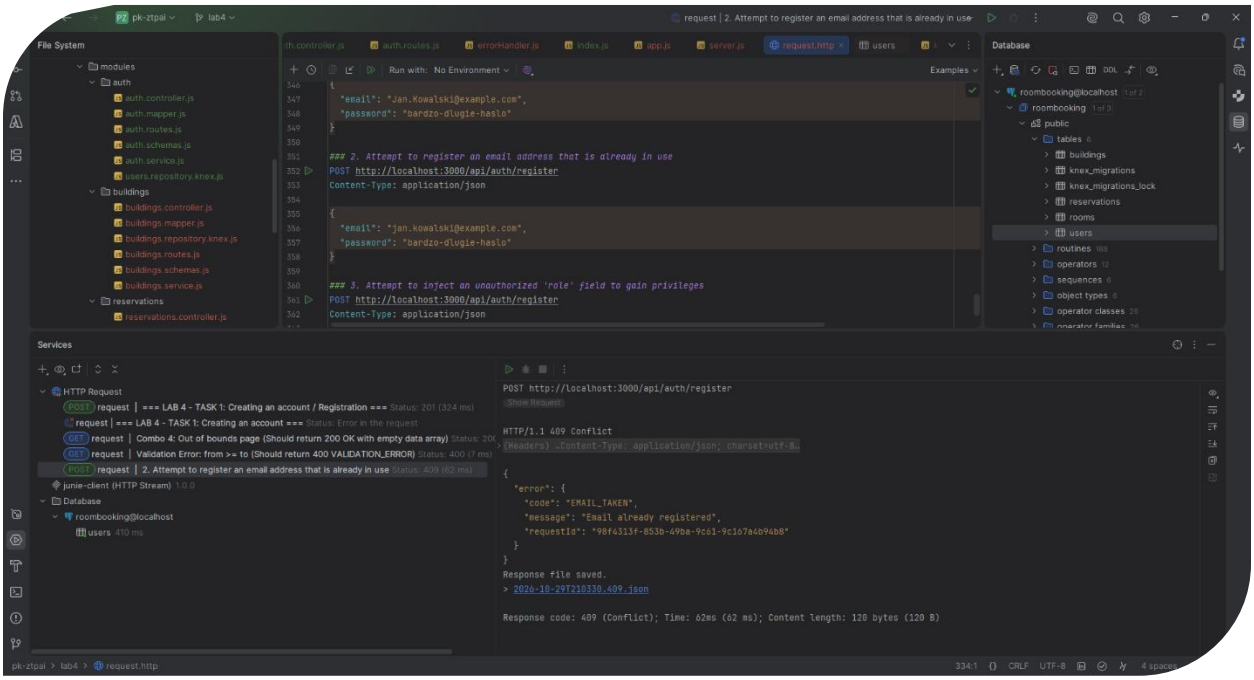
Wiersz z tabeli users w bazie danych ukazujący bezpiecznie zahashowane hasło (brak postaci jawnej) oraz znormalizowany adres e-mail.



Odpowiedź 400 Bad Request przy próbie wstrzyknięcia pola role: "ADMIN" do payloadu (zablokowane przez regułę .strict() w schemacie Zod).



Odpowiedź 400 Bad Request przy podaniu hasła krótszego niż 10 znaków.



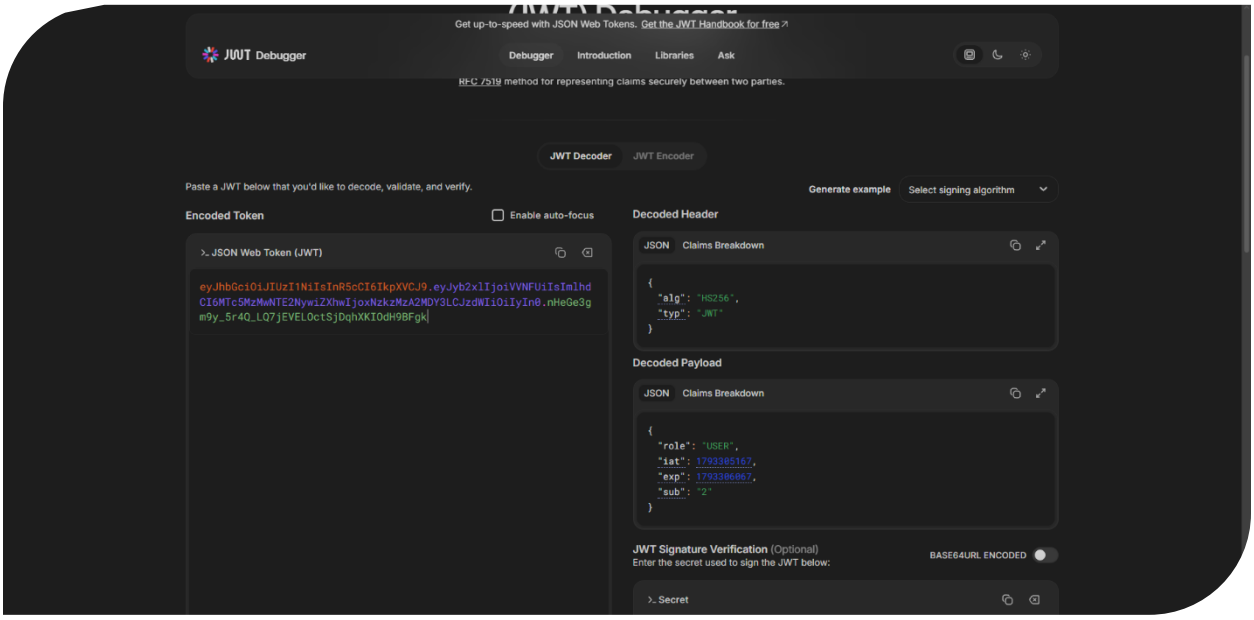
Odpowiedź 409 Conflict z kodem EMAIL_TAKEN przy próbie ponownej rejestracji na ten sam adres e-mail (inna wielkość liter).

Zadanie 2: Logowanie i JWT

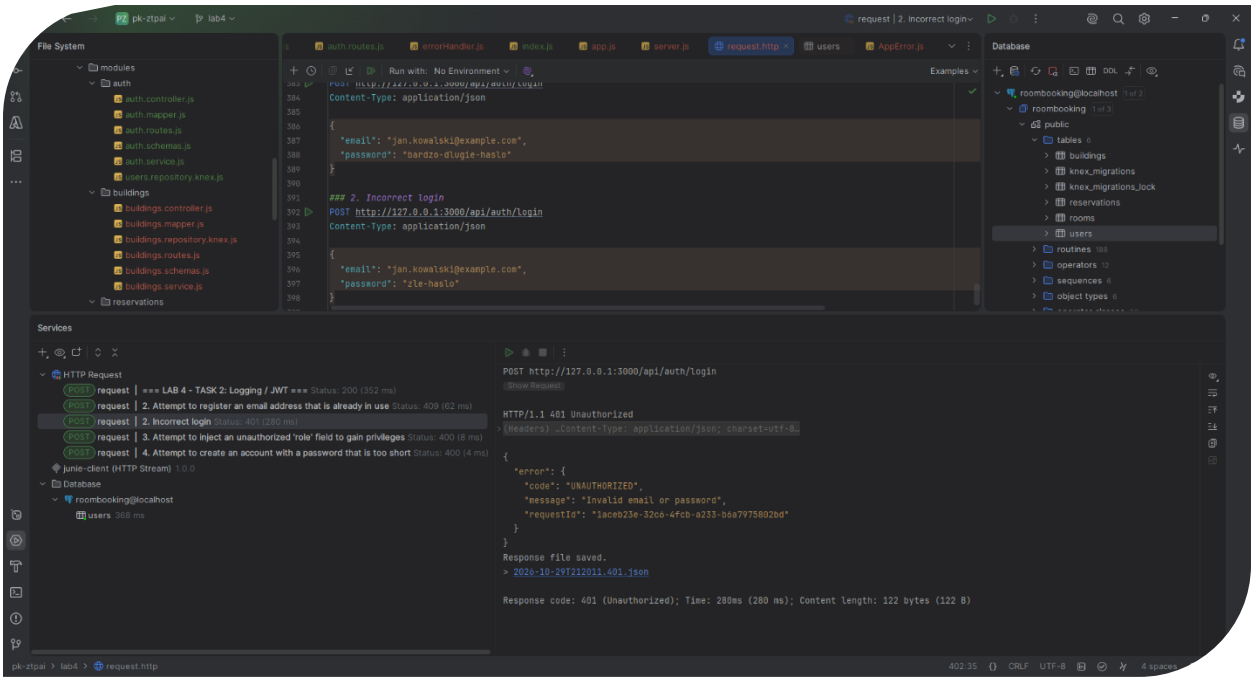
Po poprawnej weryfikacji haseł, system generuje podpisany token JWT ważny przez określony czas. Odpowiedzi serwera w przypadku błędnego loginu lub hasła są celowo identyczne, co zapobiega atakom typu *user enumeration*.

Fragment kodu (Generowanie tokenu JWT):

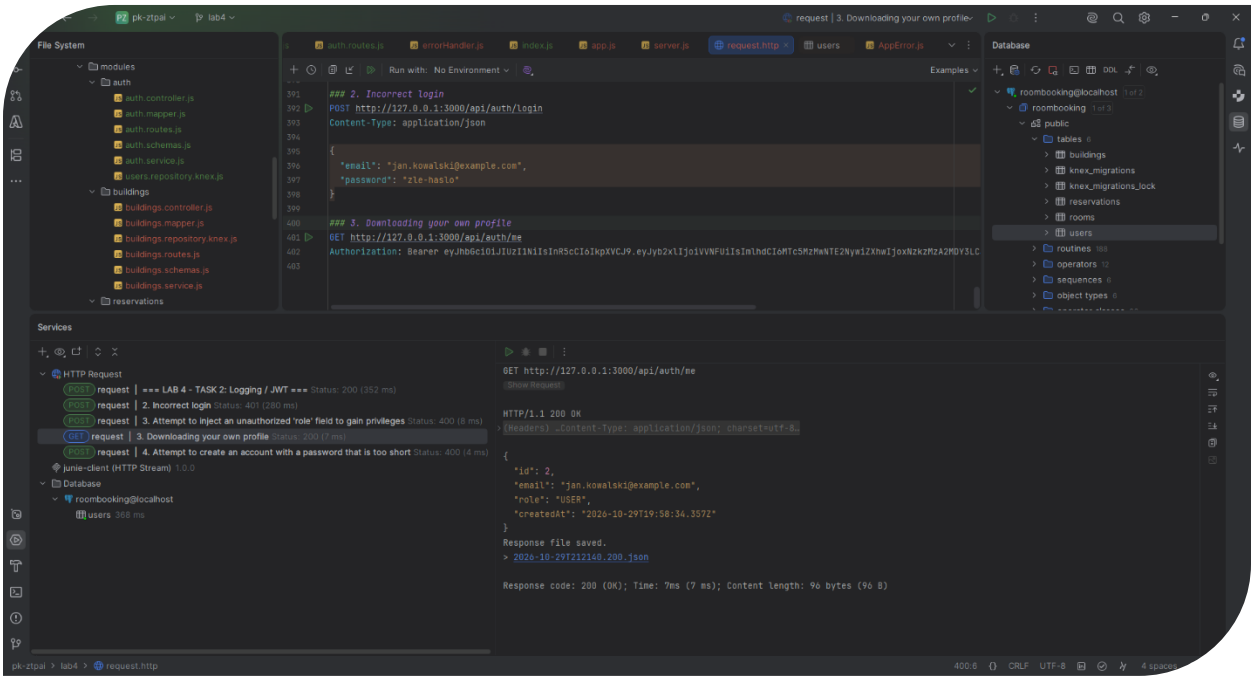
```
const accessToken = jwt.sign(
    { role: user.role },
    jwtSecret,
    { subject: String(user.id), expiresIn: jwtExpiresIn, algorithm: "HS256" }
);
```



Zdekodowany payload tokenu JWT (np. z jwt.io) demonstrujący obecność sub (ID użytkownika jako string), role, iat, exp i brak danych poufnych.



Ujednolicona odpowiedź 401 Unauthorized dla błędnych danych logowania.



Sukces pobrania własnego profilu na chronionym endpoincie /api/auth/me.

Zadanie 3: Middleware authenticate i rezerwacje użytkownika

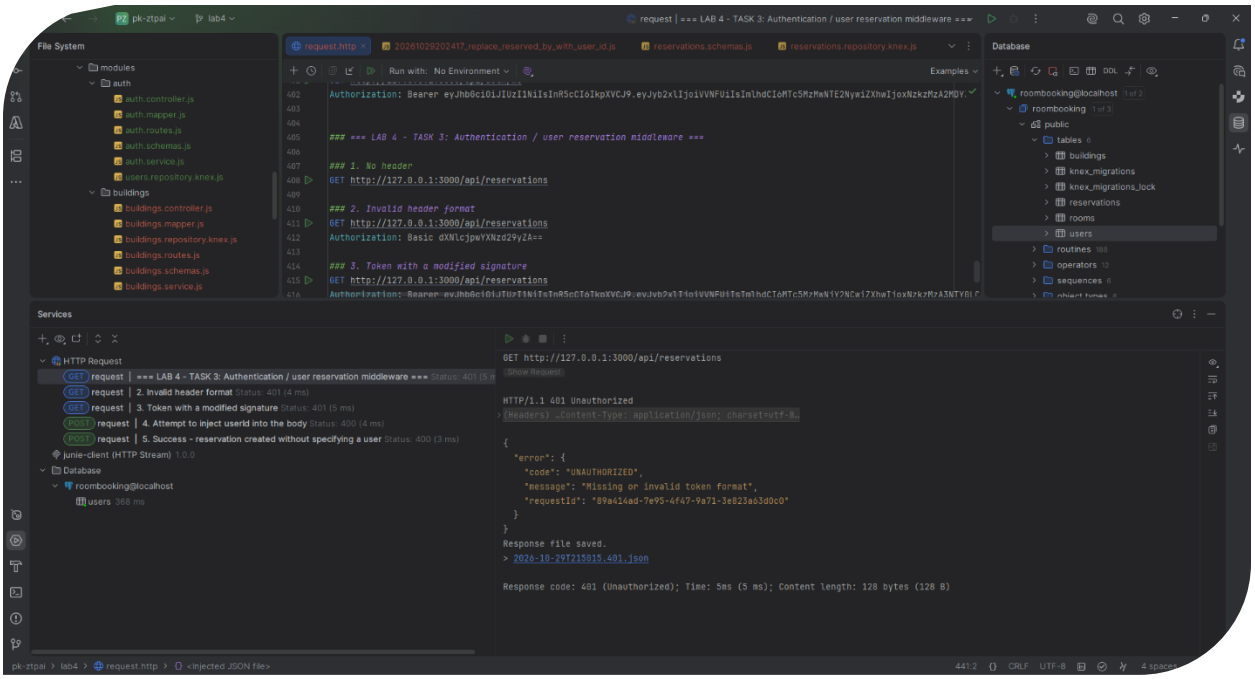
Zmieniono relację w bazie danych z wulgarnego stringa reserved_by na pełnoprawny klucz obcy user_id. Middleware weryfikuje przychodzący nagłówek i izoluje zaufaną tożsamość do obiektu req.user.

Fragment kodu (Bezpieczna migracja z zachowaniem danych):

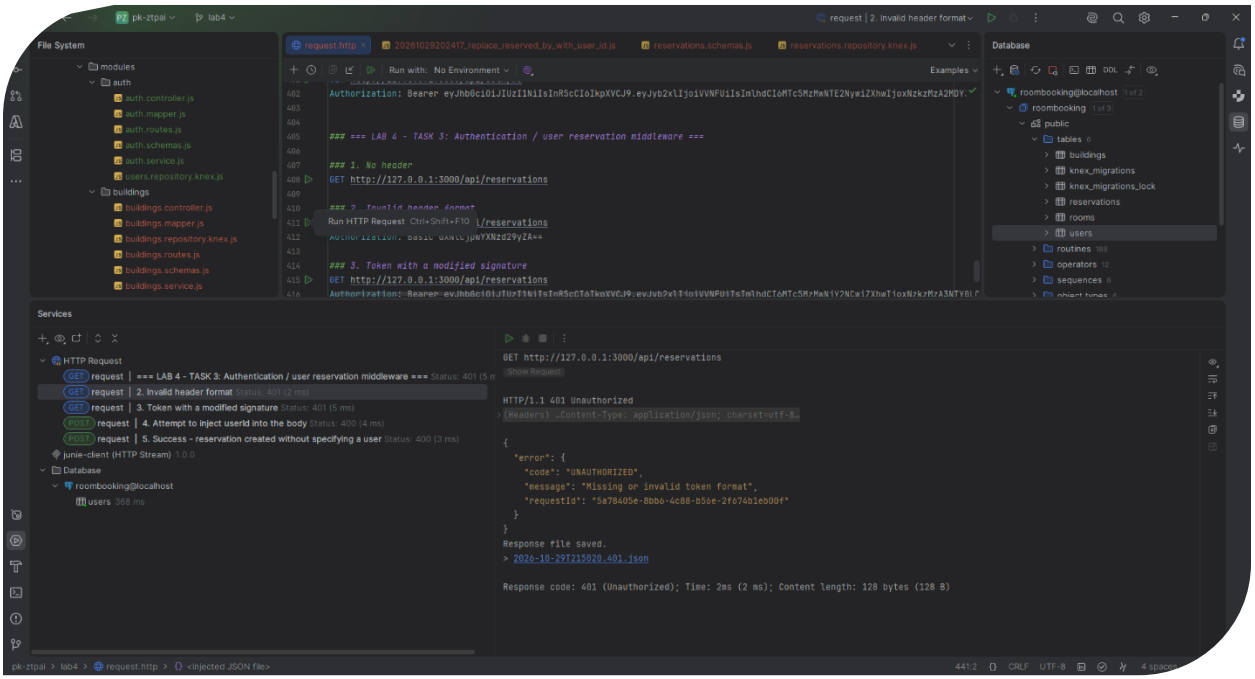
```
exports.up = async function (knex) {
    // Dodanie FK dopuszczającego NULL
    await knex.schema.alterTable("reservations", table => {
        table.integer("user_id").references("id").inTable("users").onDelete("SET NULL");
    });
    // Backfill: Przypisanie istniejących z seeda do ADMINA (ID 1)
    await knex("reservations").update({ user_id: 1 });
    // Nałożenie restrykcji NOT NULL i usunięcie starej kolumny
    await knex.schema.alterTable("reservations", table => {
        table.integer("user_id").notNullable().alter();
        table.dropColumn("reserved_by");
    });
};
```

Fragment kodu (Middleware authenticate):

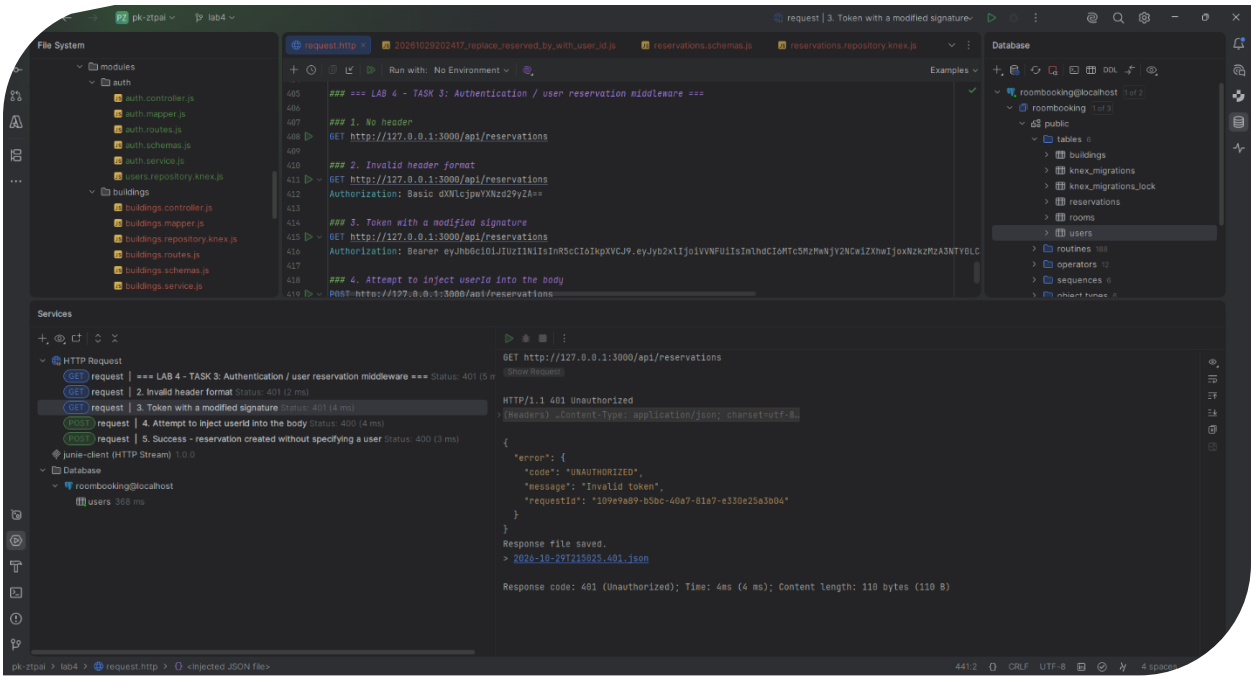
```
function authenticate(jwtSecret) {
    return (req, res, next) => {
        const header = req.get("Authorization");
        if (!header || !header.startsWith("Bearer ")) return next(new UnauthorizedError("Missing or
invalid token format"));
        // ... (weryfikacja jwt.verify i obsługa wyjątków Expired/Invalid)
        req.user = { id: Number(payload.sub), role: payload.role };
        next();
    };
}
```



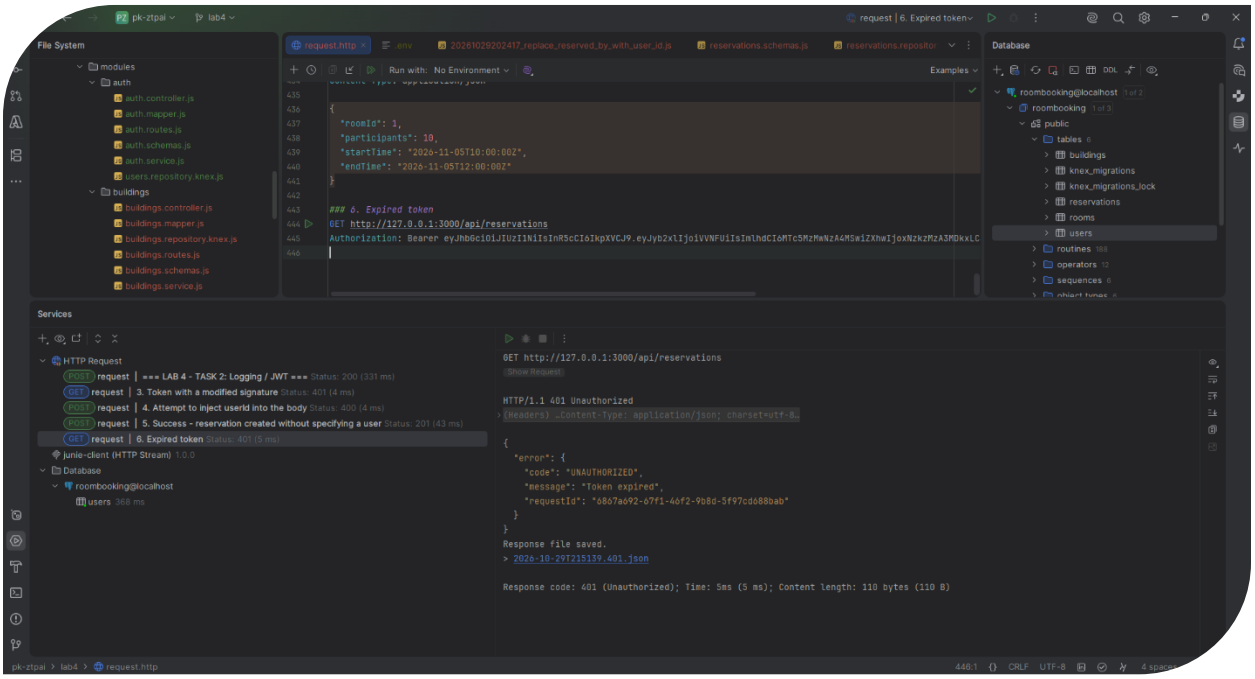
Brak nagłówka (401 Unauthorized).



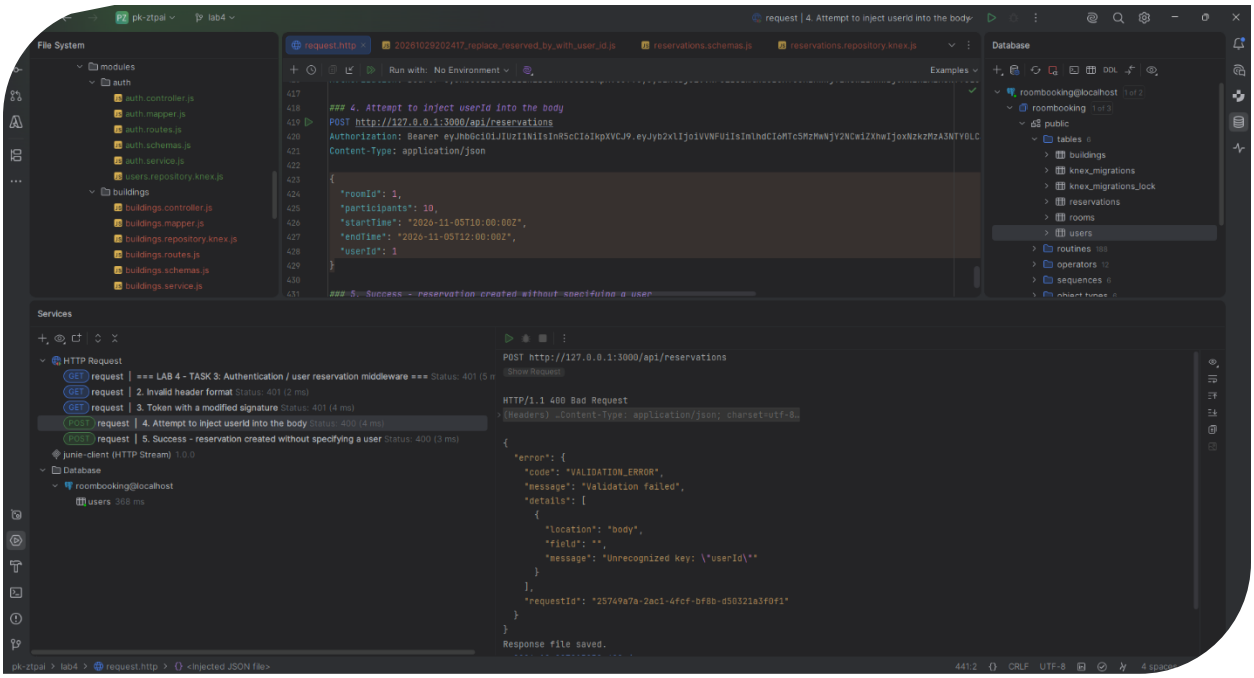
Nagłówek w złym formacie np. Basic (401 Unauthorized).



Token ze zmienioną sygnaturą (401 Unauthorized - Invalid token).



Wygasły token JWT (401 Unauthorized - Token expired).



Próba utworzenia rezerwacji z podaniem pola userId w JSON-ie (odrzucenie payloadu przez backend – ochrona przed podszywaniem się).

Zadanie 4: Authorization: role i ownership

Zaimplementowano tabelę uprawnień. Zastosowano kontrolę middleware dla endpointów modyfikujących globalne zasoby oraz kontrolę logiki biznesowej wewnątrz serwisu dla rezerwacji.

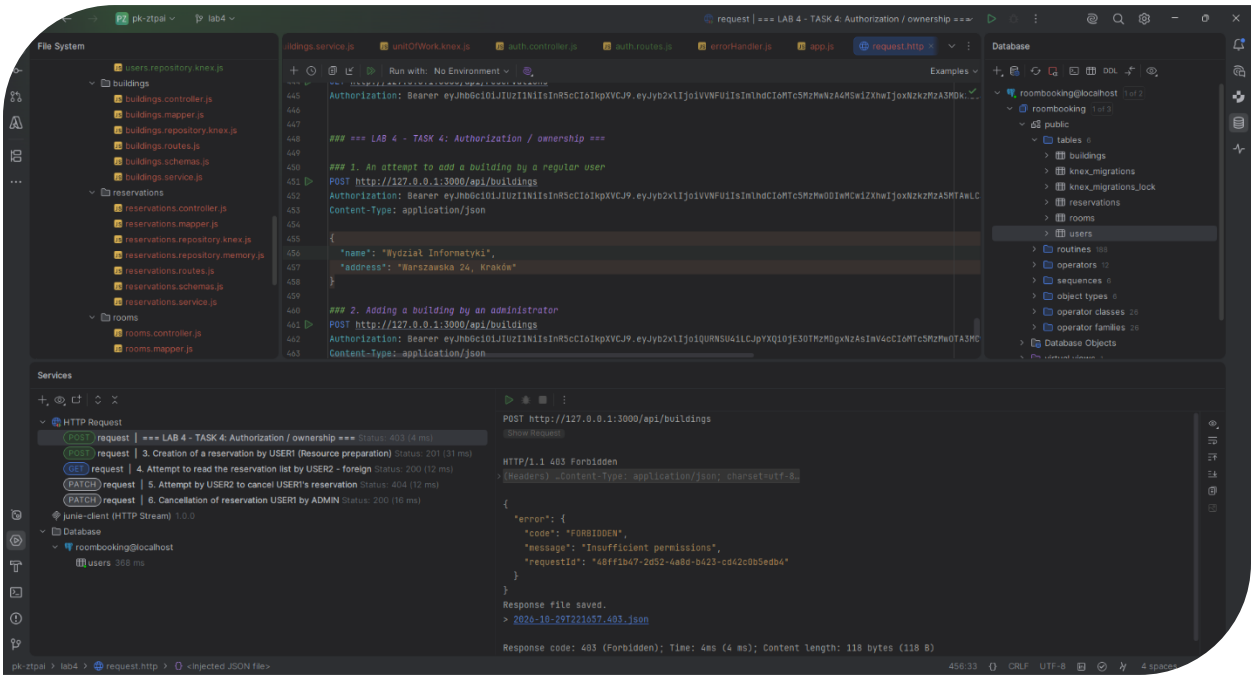
Tabela Uprawnień:

Endpoint (Akcja)	Brak Tokenu	USER (Właściciel)	USER (Obcy)	ADMIN
POST /api/buildings	401 Unauthorized	403 Forbidden	403 Forbidden	201 Created
GET /api/buildings	401 Unauthorized	200 OK	200 OK	200 OK

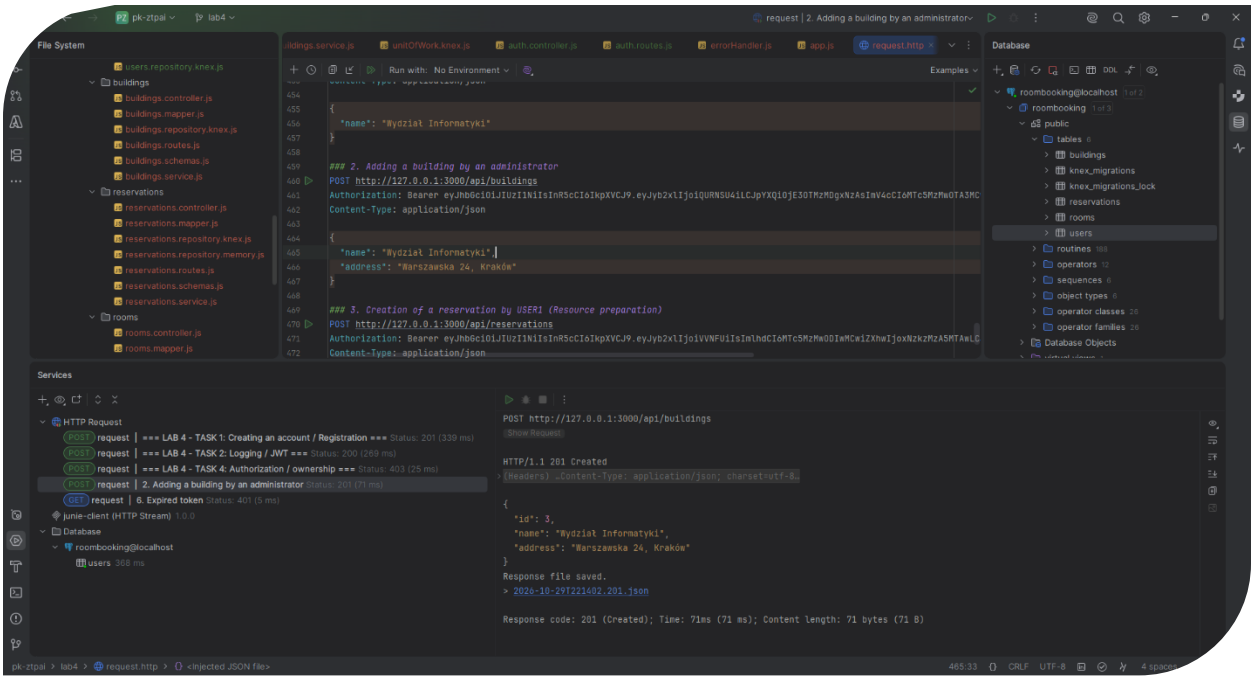
Endpoint (Akcja)	Brak Tokenu	USER (Właściciel)	USER (Obcy)	ADMIN
GET /api/reservations/:id	401 Unauthorized	200 OK	404 Not Found	200 OK
PATCH /.../cancel	401 Unauthorized	200 OK	404 Not Found	200 OK

Fragment kodu (Kontrola własności w warstwie serwisu przy odwoływaniu rezerwacji):

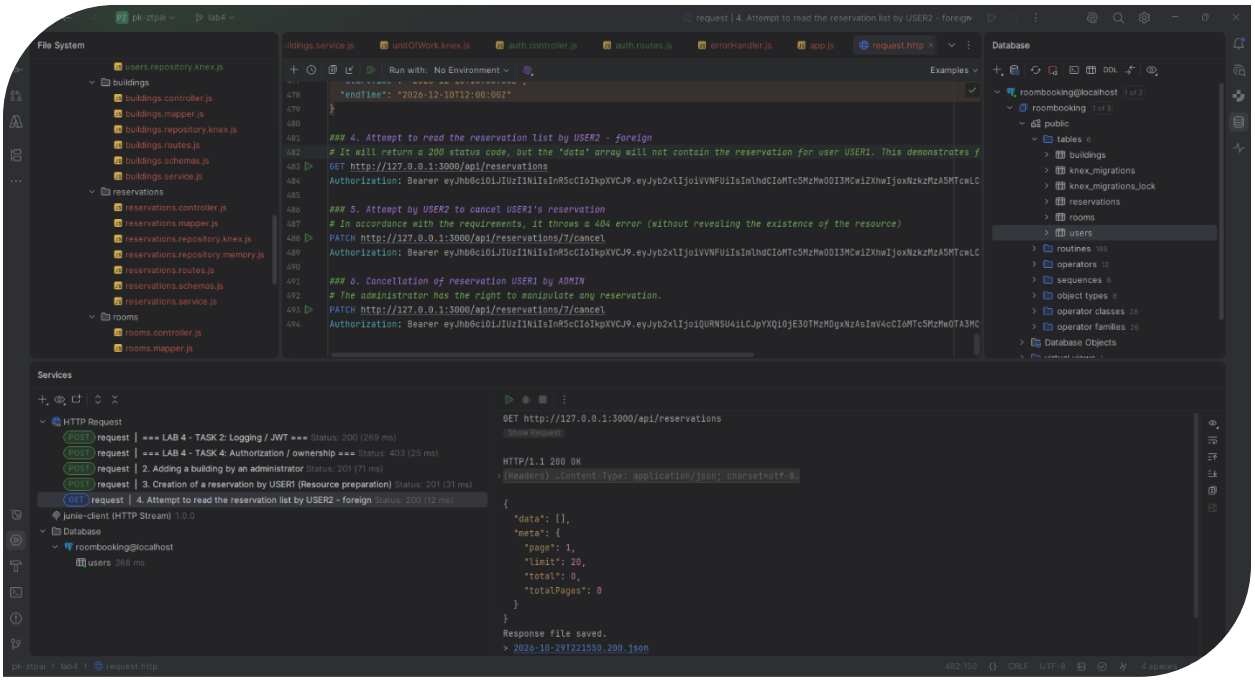
```
const isOwner = reservation.user && reservation.user.id === currentUser.id;
if (!isOwner && currentUser.role !== "ADMIN") {
    // 404 zamiast 403 by ukryć istnienie prywatnego zasobu innej osoby
    throw new NotFoundError("Reservation");
}
```



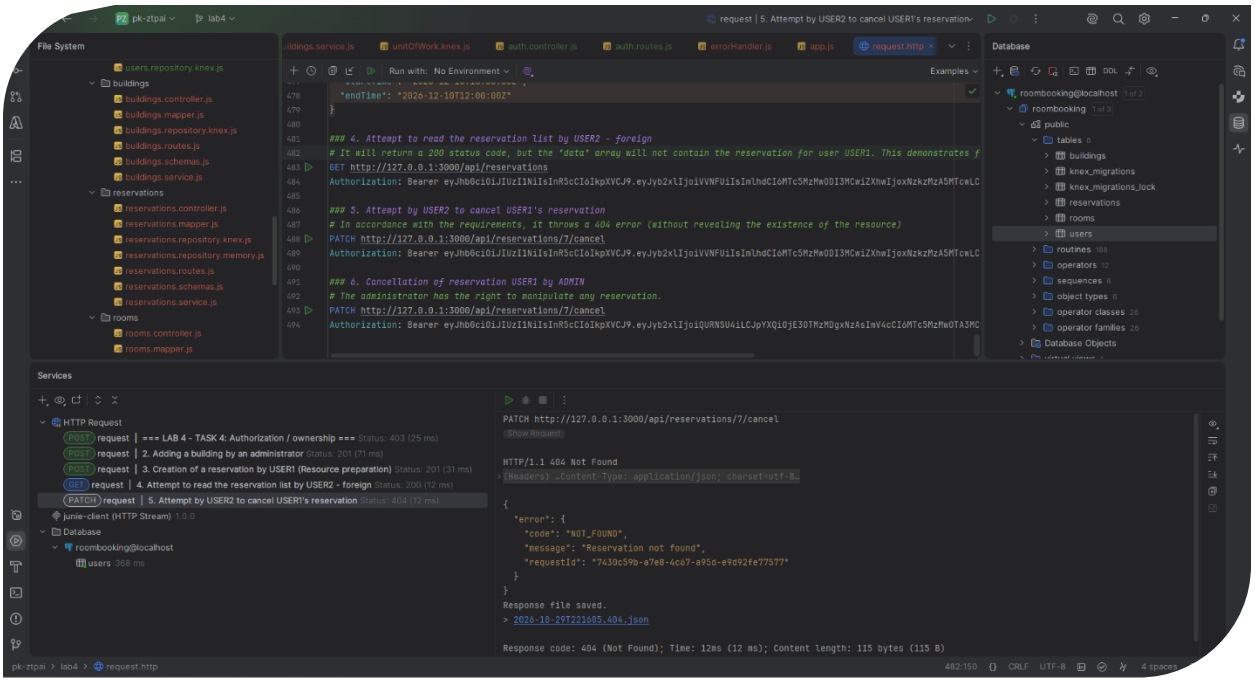
Próba utworzenia budynku przez użytkownika USER (zablokowane).



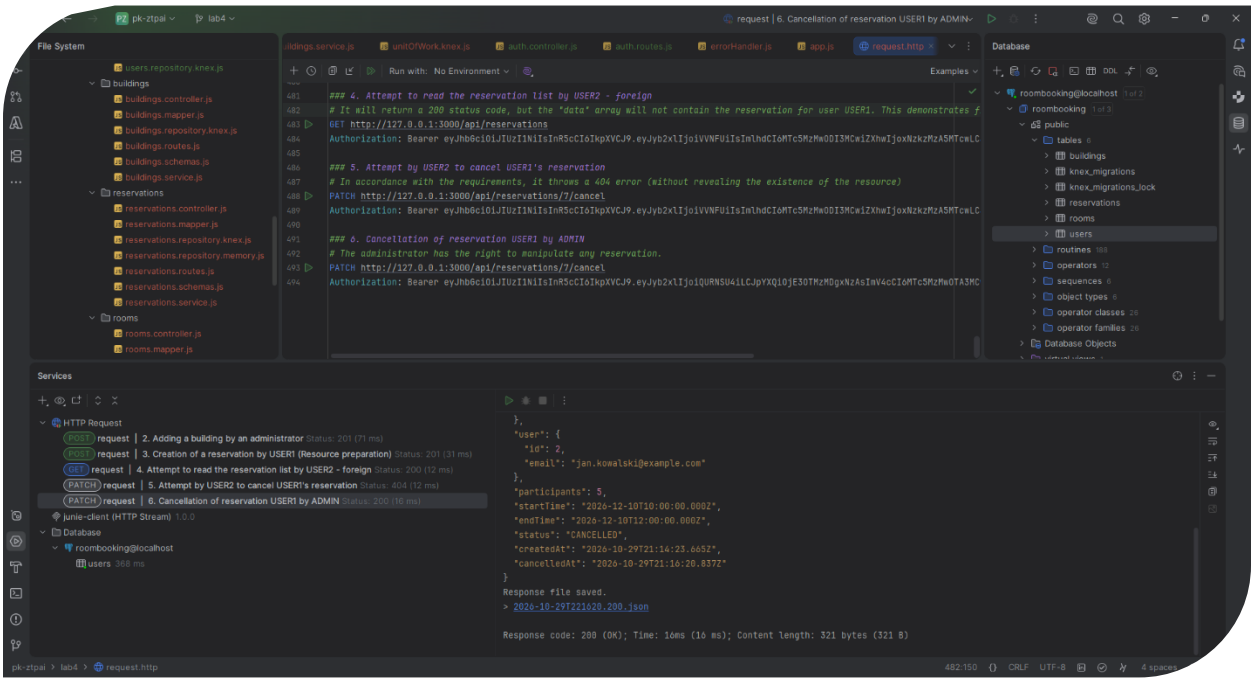
Prawidłowe utworzenie budynku przez administratora.



Pobranie listy rezerwacji przez USER2 ukazuje działający filtr backendowy – brak w wynikach rezerwacji wygenerowanej przez USER1.



Próba modyfikacji (anulowania) obcej rezerwacji przez USER2 skutkuje błędem 404.



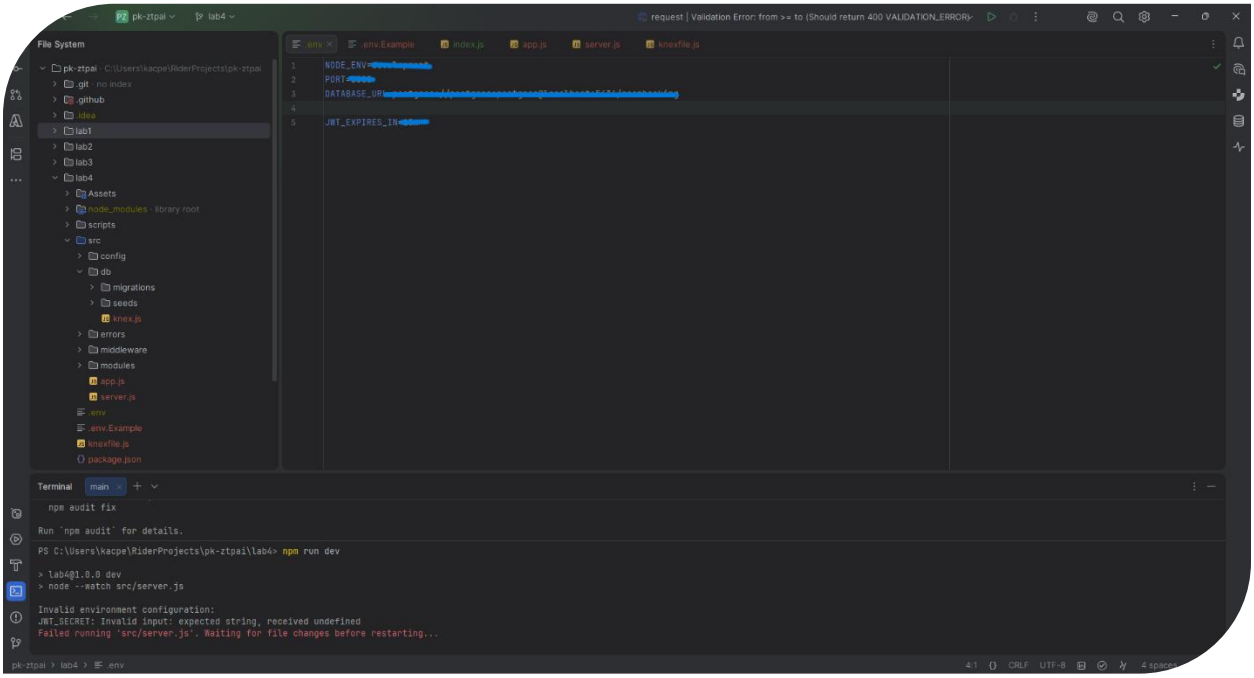
Administrator bez problemu manipuluje cudzą rezerwacją.

Zadanie 5: Konfiguracja, sekrety i dokumentacja API

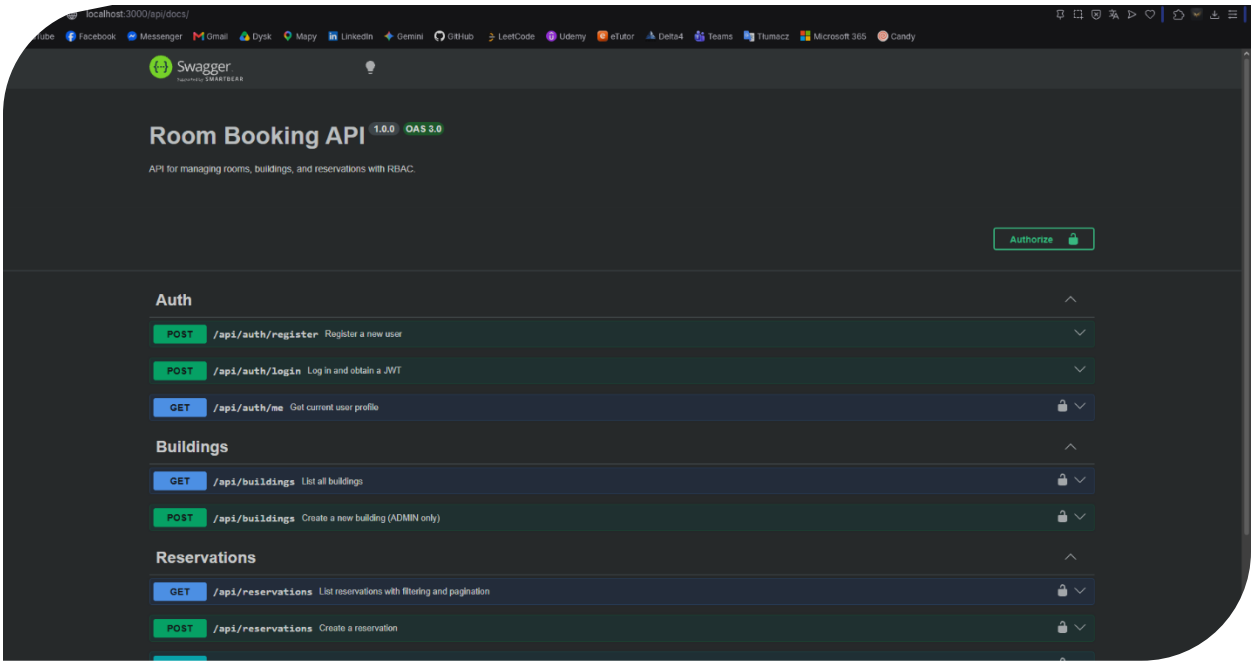
Wdrożono scentralizowany system walidacji środowiska w oparciu o bibliotekę Zod. Zabezpiecza on system przed uruchomieniem w wadliwym stanie i chroni przed brakiem klucza JWT. Zbudowano również interaktywną dokumentację w formacie OpenAPI.

Fragment kodu (Moduł konfiguracji src/config/index.js z zasadą Fail-Fast):

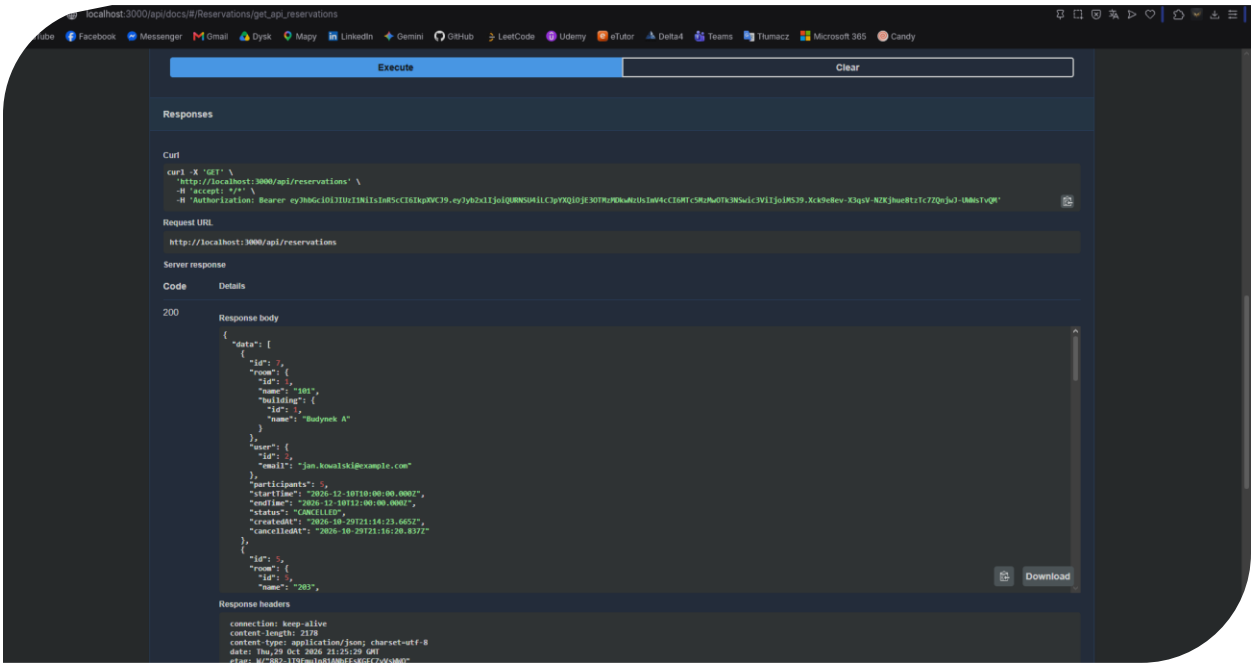
```
const envSchema = z.object({
    PORT: z.coerce.number().int().default(3000),
    DATABASE_URL: z.string().url(),
    JWT_SECRET: z.string().min(32, "JWT_SECRET must have at least 32 characters"),
    // ...
});
const result = envSchema.safeParse(process.env);
if (!result.success) {
    // Zatrzymanie serwera w przypadku błędu (Fail-Fast)
    process.exit(1);
}
```



Zrzut ekranu pokazujący request PATCH /api/reservations/1/cancel.



Zrzut ekranu pokazujący request PATCH /api/reservations/1/cancel.



Zrzut ekranu pokazujący request PATCH /api/reservations/1/cancel.